

Supplementary Material: BIM-Native Tokenization for Constraint-Aware Room Layout Synthesis

Manuel Ladron de Guevara*, Jinmo Rhee[†], Ardavan Bidgoli*, Vaidas Razgaitis*, Michael Bergin*

*Higharc, United States [†]University of Calgary, Canada

{manuelrodriguez, ardavanbidgoli, vaidasrazgaitis, michaelbergin}@higharc.com jinmo.rhee@ucalgary.ca

Accepted at EUVIP 2026, 14th European Conference on Visual Information Processing

Project page: <https://manuelladron.github.io/sbm>



Fig. 1: Small Building Model (SBM) is an encoder–decoder Transformer that generates functionally correct and semantically coherent layouts given a room envelope. In both examples, all rooms have been laid out using Data-Driven Entity Prediction (DDEP), enabled by our SBM model.

This supplementary material provides additional details on metrics, baseline adaptation, training, inference, constrained decoding, dataset statistics, and qualitative results referenced in the main paper. Sections are organized to match the order of reference in the main text.

I. LAYOUT SYNTHESIS METRICS

We evaluate synthesized layouts along three complementary axes: (i) *entity coverage*, which measures whether a layout furnishes the required inventory for its room type; (ii) *navigability*, which measures functional access from doors to key fixtures under clearance constraints; and (iii) *overlap and clearance* (OC), which quantifies collisions and door blocking based on exact geometry. Together these metrics separate *what* is present (coverage), *how* it can be used (navigability), and *how cleanly it is placed* (OC).

A. Entity Coverage

The coverage metric evaluates how well predicted layouts satisfy room-type-specific inventory requirements. In contrast to purely geometric measures (e.g., placement distance), coverage is a *combinatorial* score over categories and counts: it measures *which* entities are present, largely independent of where they are placed in the room. This is particularly important for unguided generation, where methods must decide both entity selection and placement.

1) *Specification and per-room formulation*: For each room type $r \in \mathcal{R}$ (e.g., BATHFULL, KITCHEN), we define an inventory specification $\mathcal{S}_r = \{s_1, \dots, s_n\}$ where each item s_i encodes:

- a category c_i (e.g., “sink”, “toilet”),
- a count range $[m_i^{\min}, m_i^{\max}]$ (minimum and maximum allowed),
- an essential flag $e_i \in \{0, 1\}$ (required vs. optional),
- a weight $w_i = 1.0$ if $e_i = 1$, else $w_i = 0.5$.

Given a predicted layout with entity set $\mathcal{P} = \{p_1, \dots, p_k\}$, we count how many instances of each category appear:

$$p_i = \left| \{p \in \mathcal{P} : \text{category}(p) = c_i\} \right|.$$

The coverage metric compares $\{p_i\}$ to the specification $\mathcal{S}_r$ for the corresponding room type r .

2) *Name normalization and aliasing*: Different methods use different naming conventions (e.g., DDEP emits “King”, “Queen”, “Twin”, “Full”; LLMs emit “bed”; other baselines may use “endtable” vs. “nightstand”). To enable fair comparison, we normalize all predictions to a canonical vocabulary derived from DDEP:

- 1) **DDEP normalization**. Capitalized vocab names are mapped to lowercase and collapsed to canonical categories (e.g., “King”, “Queen”, “Twin”, “Full” $\rightarrow$ “bed”).
- 2) **Alias application**. We apply a bidirectional alias mapping $\mathcal{A}$ from configuration (e.g., “nightstand” $\leftrightarrow$ “endtable”, “sofa” $\leftrightarrow$ “couch”).
- 3) **Canonical form**. After aliasing, all entity categories map to canonical names defined in $\mathcal{S}_r$.

This procedure ensures that methods are compared on *inventory*, not spelling or naming idiosyncrasies.

3) *Per-item scoring*: For each specification item s_i with predicted count p_i , we compute normalized missing and overfill fractions:

$$d_i = \frac{\max(0, m_i^{\min} - p_i)}{\max(1, m_i^{\min})} \quad (\text{missing fraction}), \quad (1)$$

$$o_i = \frac{\max(0, p_i - m_i^{\max})}{\max(1, m_i^{\max})} \quad (\text{overfill fraction}). \quad (2)$$

We then define an item score $s_i \in [0, 1]$ as

$$s_i = 1 - \text{clip}(\alpha \cdot d_i + \beta \cdot o_i, 0, 1), \quad (3)$$

with $\alpha = 1.0$ and $\beta = 0.5$. Missing required items (large d_i) are penalized more than overfilling (large o_i).

4) *Requirement groups*: Some requirements are satisfied by alternative configurations. We define requirement groups $\mathcal{G}_r = \{g_1, \dots, g_m\}$, where each group g_j encodes a set of alternative ways to satisfy a functional requirement. Examples include:

- BATHFULL.bathe: either a tub *or* a shower (one-of group),
- KITCHEN.cooking_appliance: either a single range or both an oven and a rangetop (composite group).

Each group is scored as

$$s_{g_j} = \begin{cases} 1.0 & \text{if any alternative in } g_j \text{ is satisfied,} \\ 0.0 & \text{otherwise,} \end{cases} \quad (4)$$

with weights $w_{g_j} = 1.0$ for essential groups and 0.5 for optional groups.

5) *Extraneous items penalty*: Entities predicted outside the specification are penalized explicitly. Let $\mathcal{E} = \{c : c \notin \mathcal{S}_r \text{ after aliasing}\}$ be the set of extraneous categories, and $X = \sum_{c \in \mathcal{E}} p_c$ the total extraneous count. We define

$$p_{\text{extra}} = \min \left(1, \lambda \cdot \frac{X}{\max(1, \sum_i m_i^{\min})} \right), \quad (5)$$

with $\lambda = 0.25$, scaling the penalty relative to the total required inventory. This discourages methods from populating rooms with unsupported entities (e.g., placing a shower in a half-bath).

6) *Final coverage score and interpretation*: The coverage score for a room is

$$\text{Coverage} = 100 \times \frac{\sum_i w_i \cdot s_i + \sum_j w_{g_j} \cdot s_{g_j} - p_{\text{extra}}}{\sum_i w_i + \sum_j w_{g_j}}. \quad (6)$$

Scores lie in $[0, 100]$, with 100 indicating that all required and optional requirements are satisfied within their count ranges and no extraneous items are present. We compute coverage per room and report mean $\pm$ std over rooms and runs per method and room type.

B. Room Navigability

The navigability metric evaluates whether a generated layout admits reasonable circulation from doors to semantically critical entities under a fixed clearance model. It exposes two interpretable components: (i) a reachability term that measures whether access is blocked and (ii) a detour term that measures how circuitous feasible paths are. We combine both into a single score with range $[-35, 100]$: blocked-entry rooms attain the floor score -35 , while layouts with full direct access approach 100 (see Sec. I-B3e for a worked example).

1) Design and geometric model:

a) Targets.: For each room type $r \in \mathcal{R}$ we derive a set of essential categories $\mathcal{E}_r$ from the same inventory used in the coverage metric. In bedrooms, beds contribute *two* side targets (left/right access). For other essentials (e.g., sink, toilet, range), we define a single “front” target located in front of the entity, along its outward normal.

b) Doors.: Each door opening is converted into a *door portal* by taking the midpoint of the opening and offsetting it by a fixed clearance $(r + \varepsilon)$ into the room along the inward wall normal. Duplicate openings are deduplicated. If rasterization places that candidate on a blocked cell, we allow only a tiny local snap to the nearest walkable cell (at most one grid cell). If no such correction exists, or if the immediate inward vestibule remains blocked, the door is marked invalid and all door–target pairs from that door fail.

c) Clearance and walkable space.: We model human clearance by inflating each entity’s oriented rectangle with a Minkowski sum with a disk of radius $r = w/2$, where w is the corridor width (default $w = 2.0$ ft). Inflation is performed in the entity’s local frame (not axis-aligned). The walkable region is the room polygon minus the union of all inflated obstacles. In practice we rasterize on a uniform grid; our implementation uses approximately 320 cells along the longest room side.

d) Routing.: Paths are computed on the walkable mask using A^* on an 8-connected grid. A door–target pair is deemed *reachable* if the door is valid under the doorway test above and A^* finds a path between the corresponding grid cells.

2) Target construction.: Let an entity e be wall-anchored on an edge with unit tangent $\mathbf{u}$ (along the wall) and inward unit normal $\mathbf{n}$. With local rotation θ , the entity’s local axes are

$$\mathbf{x} = \cos \theta \mathbf{u} + \sin \theta \mathbf{n}, \quad (7)$$

$$\mathbf{y} = R_{90} \mathbf{x}, \quad R_{90} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad (8)$$

where $\mathbf{x}$ spans width and $\mathbf{y}$ points into the room. Let $\mathbf{A}$ be the anchor point, w_e and d_e the width and depth (in feet), and $\varepsilon > 0$ a small numerical offset. The geometric center is

$$\mathbf{C} = \mathbf{A} + \frac{d_e}{2} \mathbf{y}.$$

Depending on semantics, e contributes side-access or front-access targets:

$$\mathcal{T}_e = \begin{cases} \{ \mathbf{C} \pm (\frac{w_e}{2} + r + \varepsilon) \mathbf{x} \}, & \text{(side access),} \\ \{ \mathbf{A} + (d_e + r + \varepsilon) \mathbf{y} \}, & \text{(front access).} \end{cases} \quad (9)$$

The set of targets for room r is $\mathcal{T}_r = \bigcup_{e \in \mathcal{E}_r} \mathcal{T}_e$.

3) Scoring.: Let $\mathcal{D}_r$ be the set of door portals and $\mathcal{T}_r$ the set of essential targets in room r . We evaluate all pairs $(d, t) \in \mathcal{D}_r \times \mathcal{T}_r$ with uniform weights.

a) Reachability.: Let $\nu(d) \in \{0, 1\}$ indicate whether door portal d is a valid entry point after the inward-offset, tiny-snap, and vestibule-clearance checks. Let $\delta(d, t) \in \{0, 1\}$ indicate whether a path exists between door portal d and target t on the walkable mask. We define pair reachability as

$$\delta_{\text{valid}}(d, t) = \nu(d) \cdot \delta(d, t).$$

The per-room success rate (SR) is

$$\text{SR}(r) = \frac{1}{|\mathcal{D}_r| |\mathcal{T}_r|} \sum_{d \in \mathcal{D}_r} \sum_{t \in \mathcal{T}_r} \delta_{\text{valid}}(d, t). \quad (10)$$

Intuitively, SR measures the fraction of door–target pairs that are reachable; $\text{SR} = 1$ indicates that every essential target can be reached from every door without collision.

b) *Detour factor.*: For reachable pairs (d, t) , let $L(d, t)$ be the shortest-path length and $E(d, t) = \|\mathbf{t} - \mathbf{d}\|_2$ the Euclidean distance. We define the detour ratio $\rho(d, t) = L(d, t) / \max(E(d, t), \epsilon) \geq 1$, with $\epsilon > 0$ to avoid division by zero. Detours are mapped to $[0, 1]$ with a $3\times$ -overtravel cap:

$$f(\rho) = \min\left(1, \frac{\rho - 1}{3 - 1}\right). \quad (11)$$

The per-room detour penalty is

$$\text{DF}(r) = \begin{cases} \frac{1}{\sum_{d,t} \delta_{\text{valid}}(d, t)} \sum_{d,t} \delta_{\text{valid}}(d, t) f(\rho(d, t)), & \text{if } \sum_{d,t} \delta_{\text{valid}}(d, t) > 0, \\ 1, & \text{otherwise,} \end{cases} \quad (12)$$

averaged over reachable pairs only.

c) *Final score.*: We combine SR and DF with a trade-off weight λ (default $\lambda = 0.35$):

$$\text{Nav}(r) = 100 \cdot (\text{SR}(r) - \lambda \text{DF}(r)). \quad (13)$$

Unreachable pairs reduce SR; detours penalize only reachable pairs, avoiding double counting.

d) *Score range and boundary behavior.*: When no door–target pair is reachable ($\text{SR} = 0$), DF has no reachable pairs to average over and defaults to its worst value 1, yielding $\text{Nav} = -100\lambda$. When at least one pair is reachable, DF can be as high as 1.0 (all reachable paths suffer $\geq 3\times$ detour), so Nav can become negative. The algebraic range is $\text{Nav} \in (-100\lambda, 100]$, i.e. $(-35, 100]$ with the default $\lambda=0.35$. The lower bound is achieved when no pair is reachable and the fallback worst-case detour penalty is applied.

e) *Interpretation: why negative scores occur.*: A negative Nav score indicates severe functional failure. This can happen in two ways: either no door–target pair is reachable (yielding the floor score -35), or a few pairs are reachable but so circuitous that the detour penalty outweighs the reachability reward. Concretely, Nav turns negative when $\text{SR} < \lambda \cdot \text{DF}$.

f) *Worked example from Table I in main paper.*: Consider a room with 2 doors and 4 essential targets (8 door–target pairs).

Method	SR	DF	Nav	Interpretation
DDEP	0.36	0.60	+14.6	partial reachability with moderate detours
ATISS	0.03	0.97	−30.3	almost no reachable pairs, near-worst detours
BLT	0.12	0.88	−18.6	sparse reachability with severe detours
Blocked room	0.00	1.00	−35.0	no reachable pairs $\Rightarrow$ worst-case detour fallback

For ATISS ($\text{Nav} = -30.3$): the model leaves almost all essential targets unreachable, and the few surviving paths incur near-worst detours. Using the shared-subset Table I values, $100 \times (0.03 - 0.35 \times 0.97) \approx -30.3$. This sits close to the blocked-room floor score of -35 , indicating that the layout is functionally unusable.

C. Overlap and Clearance (OC)

The OC metrics quantify geometric collisions and door blocking using exact polygonal geometry. They complement coverage and navigability: coverage can be high even if items overlap; navigability can be positive even if some non-essential objects intersect. OC explicitly measures such violations.

Let $R \subset \mathbb{R}^2$ be the room polygon with area $A_R = \text{area}(R)$, and $\{P_i\}_{i=1}^N$ the oriented footprint polygons of entities, all in feet. Each entity has a type (e.g., PROP, CASEWORK) and category (e.g., base_cabinet, sofa).

We define an eligibility predicate for counting overlaps:

$$\text{elig}(i, j) \iff (\text{type}(i) = \text{type}(j) = \text{PROP}) \vee (\text{type}(i) = \text{type}(j) = \text{CASEWORK} \wedge \text{cat}(i) = \text{cat}(j)). \quad (14)$$

That is, we count overlaps among props, and among same-category casework (e.g., base cabinets with base cabinets), but ignore overlaps between props and casework, which often correspond to intentional placements (e.g., a lamp on a nightstand).

a) *Entity overlap fraction (EOF).*: For each entity i , we compute its overlapped area with eligible neighbors:

$$O_i = \text{area}\left(P_i \cap \bigcup_{j \neq i, \text{elig}(i,j)} P_j\right), f_i = \min\left(1, \frac{O_i}{\text{area}(P_i)}\right).$$

Aggregating over entities (optionally weighting essentials by w_i),

$$\text{EOF} = \frac{\sum_i w_i f_i}{\sum_i w_i}.$$

b) *Global overlap area (GOA)*.: Let $\Omega = \bigcup_{i < j, \text{elig}(i,j)} (P_i \cap P_j)$ be the union of all overlapped regions induced by eligible pairs. We normalize by room size:

$$\text{GOA} = \frac{\text{area}(\Omega)}{A_R}.$$

c) *Door clearance intrusion (DCI)*.: For each door d , we construct an inward clearance zone C_d by extruding the door segment into the room along the inward normal by depth c_d (default $c_d=3.0$ ft). The per-door intrusion is

$$g_d = \frac{\text{area}(C_d \cap \bigcup_{i=1}^N P_i)}{\text{area}(C_d)}, \quad \text{DCI} = \frac{1}{D} \sum_{d=1}^D g_d.$$

d) *Wall-bounds violation (WBV)*.: For each entity i with footprint P_i , we compute the violation fraction

$$v_i = \frac{\text{area}(P_i \setminus R)}{\text{area}(P_i)}, \quad \text{WBV} = \frac{1}{N} \sum_{i=1}^N v_i.$$

e) *Combined OC score*.: The composite OC percentage reported in the main paper combines three components:

$$\text{OC} (\%) = 100 \times (w_{\text{EOF}} \text{EOF} + w_{\text{GOA}} \text{GOA} + w_{\text{DCI}} \text{DCI}), \quad (15)$$

where $w_{\text{EOF}} = 0.5$, $w_{\text{GOA}} = 0.2$, and $w_{\text{DCI}} = 0.3$. Entity overlaps are weighted most heavily, followed by door clearance intrusions and global overlap area. WBV is computed per entity and reported in detailed per-room metrics but is excluded from the composite score because wall-boundary violations are strongly correlated with EOF (entities protruding outside the room also register as overlapping with the room boundary).

II. BASELINE ADAPTATION PIPELINE

The controlled same-data benchmark (Sec. IV-B in the main paper) compares DDEP against ATISS [1] and BLT [2], which use different native output representations. Both baselines operate on axis-aligned bounding boxes in normalized or global coordinates, whereas DDEP produces wall-referenced entities with (`edge_id`, `t_value`, `lateral_offset`). This section details the full pipeline that (i) converts our DDEP training data into each baseline’s native format, (ii) trains each baseline on the *identical* data split, and (iii) reprojects their predictions back into wall-referenced coordinates for unified metric computation.

All three methods share a frozen data split of 15,740 / 2,009 / 1,826 rooms (train / val / test) across five room types: BATHFULL, BEDROOM, LAUNDRY, LIVING, and MASTER_BED. The split is pinned by a deterministic manifest with per-room SHA-256 checksums to guarantee reproducibility. For current scoring, all methods are evaluated on the 1,231-room DDEP-valid subset of that test split, and direct cross-method comparisons use the shared 1,225-room intersection where every method materializes an output.

A. ATISS Adaptation

1) *Data conversion: DDEP to ATISS format*: ATISS expects a *cached_threedfront* dataset consisting of per-room `boxes.npz` archives with arrays for class labels, 3D translations, half-sizes, angles, a room floor plan (mesh), and a rasterized room mask. Our conversion proceeds in two stages.

a) *Stage 1: Official-input export*.: We export each DDEP training room to an intermediate *official-input* bundle containing:

- A room mask (256×256 binary PNG) rendered by rasterizing the room polygon with 8-pixel padding,
- A per-room JSON with entity bounding boxes in globally-normalized coordinates,
- Split-level JSONL manifests (`train.jsonl`, `val.jsonl`, `test.jsonl`).

Room geometry uses the *global* normalization scheme: edges come from the dataset’s `segment_map_ordered_norm_global` field, preserving cross-room scale consistency. Entity positions and dimensions are converted from DDEP’s inch-based representation to this global-normalized coordinate frame via a per-room scale factor:

$$s = \frac{\text{median}_e(\ell_e^{\text{raw}} / \ell_e^{\text{norm}})}{12}, \quad (16)$$

where ℓ_e^{raw} is the raw edge length in inches and ℓ_e^{norm} is the normalized edge length for the same edge e . This recovers the feet-per-normalized-unit scale, and entity dimensions are divided by $12s$ to convert from inches to normalized units.

b) *Stage 2: ATISS cached format.*: The official-input bundle is then converted to ATISS’s `cached_threedfront` format. For each room, we produce a `boxes.npz` with:

- `class_labels`: one-hot vectors of shape $(N, C+2)$, where C is the number of furniture classes and 2 accounts for start/end tokens,
- `translations`: $(N, 3)$ in normalized room coordinates, with the y-axis fixed at 0.5 (floor height),
- `sizes`: $(N, 3)$ as *half*-widths, not full dimensions (ATISS convention),
- `angles`: $(N, 1)$ in radians,
- `floor_plan_vertices` / `floor_plan_faces`: triangulated room polygon mesh,
- `room_layout`: $(256, 256, 1)$ rasterized room mask.

We also compute *dataset statistics* (`dataset_stats.txt`) over the training and validation splits, recording bounding ranges for translations, sizes, and angles. These bounds define the normalization range used by ATISS’s DMLL (Discretized Mixture of Logistics Layers) during training and are critical for correct denormalization at inference time.

c) *Category mapping.*: Entity categories are canonicalized to DDEP’s vocabulary using a multi-step resolution: (1) direct string match against the decoder vocabulary, (2) case-insensitive + whitespace-normalized match, (3) special alias rules (e.g., any name containing “bath”/“tub”/“shower” maps to `Tub`), (4) fallback to a `vocab_mapping.json` that maps raw DDEP labels to canonical names. Unresolvable labels are assigned `<UNK>`.

2) *Training configuration.*: We train ATISS using its published autoregressive transformer architecture with no method-specific tuning. Table I lists all hyperparameters.

TABLE I: ATISS training hyperparameters. All values match the published defaults except batch size and learning rate, which are scaled together ($4\times$ linear scaling from the original 128/0.0001 to 1024/0.0004).

Parameter	Value
Architecture	Autoregressive Transformer
Layers / Heads	4 / 8
Hidden dimension	512
Query / Value dimensions	64 / 64
Feed-forward dimension	1024
Number of DMLL mixtures	10
BBox output type	<code>autoregressive_mlc</code>
Feature extractor	ResNet-18 (64-dim, 1-channel input)
Epochs	300
Steps per epoch	500
Batch size	1024
Optimizer	Adam
Learning rate	4×10^{-4}
Weight decay	0.0
Checkpoint frequency	Every 25 epochs
Validation frequency	Every 5 epochs
Random seed	17
Room mask resolution	256×256
GPU	NVIDIA H100

ATISS conditions entity generation on the room mask via a frozen-BN ResNet-18 encoder (1-channel input, 64-dimensional output). It autoregressively generates entities by sampling class labels, translations, sizes, and angles from a DMLL distribution. Generation stops at the end-of-sequence (EOS) token.

3) *Inference and denormalization.*: At inference time, ATISS produces per-entity predictions in its internal normalized representation $x \in [-1, 1]$, which must be mapped back to physical coordinates. The denormalization follows the DMLL descale formula:

$$\hat{v} = \frac{x+1}{2} (v_{\max} - v_{\min}) + v_{\min}, \quad (17)$$

where $v_{\min}$ and $v_{\max}$ are the per-axis bounds from the dataset statistics computed during data preparation.

This formula is applied independently to three quantities:

- 1) **Translations** (center position). ATISS uses a Y-up 3D coordinate system $[x, y_{\text{height}}, z]$; we extract x and z (the floor-plan axes) and discard height. After Eq. (17), positions are in the globally-normalized room coordinate frame.
- 2) **Sizes** (half-extents). ATISS stores *half*-widths; after denormalization, we multiply by 2 to recover full dimensions and then by 12 to convert from feet to inches:

$$w_{\text{in}} = 2 \times 12 \times \hat{s}_w, \quad d_{\text{in}} = 2 \times 12 \times \hat{s}_d. \quad (18)$$

3) **Angles.** Stored in radians; after denormalization the result is converted to degrees modulo 360° .

4) *Reprojection to wall-referenced coordinates:* ATISS predictions are axis-aligned bounding boxes with absolute centers (c_x, c_y) , whereas our evaluation metrics require wall-anchored entities with $(\text{edge_id}, \text{t_value}, \text{lateral_offset})$. We convert via *nearest-wall projection*: for each entity center, we find the closest room edge and project the center onto that edge.

Given entity center $\mathbf{p} = (c_x, c_y)$ and a room edge from $\mathbf{a}$ to $\mathbf{b}$, define $\mathbf{v} = \mathbf{b} - \mathbf{a}$ and:

$$t_{\text{raw}} = \frac{(\mathbf{p} - \mathbf{a}) \cdot \mathbf{v}}{|\mathbf{v}|^2}, \quad t = \text{clamp}(t_{\text{raw}}, 0, 1), \quad (19)$$

$$\mathbf{q} = \mathbf{a} + t\mathbf{v}, \quad d = |\mathbf{p} - \mathbf{q}|, \quad (20)$$

$$\ell = \frac{\mathbf{v} \times (\mathbf{p} - \mathbf{a})}{|\mathbf{v}|}, \quad (21)$$

where t is the normalized position along the edge, d is the perpendicular distance (used to select the nearest edge), and ℓ is the signed lateral offset (positive when the entity is to the left of the edge direction).

The entity is assigned to the edge with smallest d , producing $(\text{edge_id}, \text{t_value}=t, \text{lateral_offset}=\ell)$.

a) *Dimension override.*: To ensure evaluation fairness, predicted widths and depths are replaced with DDEP’s canonical dimension dictionary when a matching category exists. This prevents ATISS’s dimension predictions (which may drift due to the different representation) from unfairly penalizing or inflating overlap/clearance scores. When no canonical match is found and the predicted dimensions are within $[2, 200]$ inches, the predicted values are retained; otherwise, hardcoded room-type-specific heuristic fallback dimensions are applied (e.g., 36×36 inches for BEDROOM, 24×20 inches for BATHFULL). These fallbacks are conservative mid-range estimates chosen to represent generic furniture footprints for each room type; they fire only for entities with opaque asset IDs that cannot be resolved against the canonical dictionary.

B. BLT Adaptation

1) *Data conversion: DDEP to BLT format:* BLT [2] targets 2D graphic-design layout and operates on discretized (c, w, h, x, y) tokens representing category, width, height, and center coordinates, all in a normalized $[0, 1]$ coordinate system relative to the canvas (room) bounds.

a) *Format.*: We convert from the ATISS official-input format to BLT’s JSON schema. Each training sample is:

```
{ "id": "room_uid", "width": W, "height": H,
  "children": [ { "category_id": 3, "center": [0.45, 0.62],
                  "width": 0.12, "height": 0.08 }, ... ] }
```

where width/height are room dimensions and each child’s center and extent are expressed relative to the room bounding box. All bounding boxes are clipped to $[0, W] \times [0, H]$ prior to normalization.

b) *Discretization.*: BLT discretizes each continuous coordinate into 32 bins (resolution $R = 32$), producing integer tokens:

$$\text{bin}(v) = \text{clamp}(\lfloor v \cdot R \rfloor, 0, R-1). \quad (22)$$

Each entity is thus encoded as a 5-token group:

$[\text{class_id}, \text{bin}(w), \text{bin}(h), \text{bin}(c_x), \text{bin}(c_y)]$.

c) *Category mapping.*: The same multi-step resolution procedure as ATISS (Section II-A) is used to map DDEP categories to a contiguous integer space. A `class_map.json` records the bidirectional label $\leftrightarrow$ ID mapping. The effective vocabulary size is $C + 132$, where C is the number of furniture classes; the additional 132 tokens cover 4 special tokens (`<PAD>`, `<SOS>`, `<EOS>`, `<UNK>`) plus 4×32 discretized coordinate bins.

2) *Training configuration:* We use BLT’s published *Transformer* (encoder–decoder seq2seq) architecture. Table II lists all hyperparameters.

a) *Runtime compatibility.*: BLT’s published codebase uses the legacy `flax.optim` API, which is deprecated in modern Flax releases. We provide a thin runtime shim that wraps BLT’s optimizer calls with equivalent Optax-based updates (`scale_by_adam` $\rightarrow$ `apply_gradient`), dataset registration patches to accept our custom class vocabulary instead of BLT’s built-in dataset enums (RICO, COCO), and import-path aliases to resolve inconsistencies between BLT’s `layout_blt` and `layout_gvt_public` module names. No architectural or loss modifications are made.

3) *Inference and decoding:* BLT generates entity tokens autoregressively. Each 5-token group is decoded as:

$$\hat{w} = \text{clamp}\left(\frac{\text{bin}_w}{R-1}, 0, 1\right), \quad \hat{h} = \text{clamp}\left(\frac{\text{bin}_h}{R-1}, 0, 1\right), \quad (23)$$

$$\hat{c}_x = \text{clamp}\left(\frac{\text{bin}_{c_x}}{R-1}, 0, 1\right), \quad \hat{c}_y = \text{clamp}\left(\frac{\text{bin}_{c_y}}{R-1}, 0, 1\right), \quad (24)$$

where $R = 32$. These normalized values are then scaled to room dimensions.

TABLE II: BLT training hyperparameters. All values follow the published defaults. The sequence length cap of 128 tokens accommodates up to 25 entities per room (5 tokens each, plus special tokens).

Parameter	Value
Architecture	Transformer (encoder–decoder)
Framework	JAX / Flax
Training steps	30,000
Batch size	64
Max sequence length	128 tokens
Evaluation frequency	Every 1,000 steps
Checkpoint frequency	Every 5,000 steps
Coordinate resolution	32 bins per axis
Random seed	17
GPU	NVIDIA H100

a) *Size clamping.*: BLT occasionally decodes degenerate boxes where $\hat{w}$ or $\hat{h}$ approaches 1.0, filling the entire room. We clamp all decoded dimensions to at most 50% of the corresponding room extent:

$$w_{\text{clamped}} = \min(\hat{w}, 0.5), \quad h_{\text{clamped}} = \min(\hat{h}, 0.5). \quad (25)$$

4) *Reprojection to wall-referenced coordinates*: BLT predictions are reprojected to wall-referenced coordinates using the same nearest-wall projection described in Section II-A4. The normalized BLT centers $(\hat{c}_x, \hat{c}_y)$ are first scaled to physical room coordinates (inches) using the room perimeter ratio:

$$s_{\text{BLT}} = \frac{P_{\text{inches}}}{P_{\text{norm}}}, \quad (26)$$

where P_{inches} is the room perimeter from DDEP metadata and P_{norm} is the perimeter of the normalized edge polygon. After scaling, the same edge projection and dimension override procedures from Section II-A4 are applied.

a) *Entity type classification.*: BLT does not distinguish entity types natively. We classify predicted entities as CASEWORK if their category name contains “cabinet”, “shelf”, or “casework”; all others are classified as PROP.

C. Validation of Representational Bridges

The nearest-wall projection and dimension override pipeline are shared identically between ATISS and BLT, ensuring that any geometric artifacts introduced by the conversion affect both baselines equally. Two observations clarify how to interpret the same-data results:

- 1) **OC must be read together with coverage.** On the shared controlled subset, ATISS achieves 0.5% OC, DDEP achieves 13.4%, and BLT achieves 35.7% (Table I in the main paper). ATISS’s near-zero OC is not evidence of superior placement quality; it is largely driven by sparse or nearly empty predictions, which trivially avoid overlaps and door intrusions. BLT’s much higher OC reflects genuine geometric failure.
- 2) **Dominant gap is in inventory and reachability.** DDEP’s advantage over baselines remains clearest in coverage and navigability: 54.9% coverage and 14.6 Nav, versus ATISS at 40.8% and −30.3, and BLT at 17.8% and −18.6. These metrics operate on entity *identity* and *reachability*—properties that are invariant to the specific wall-projection parameters. The performance gap is thus attributable to the models themselves, not the representational bridge.

a) *No method-specific tuning.*: Both ATISS and BLT use their published default hyperparameters (Tables I and II). The only deviation from published defaults is ATISS’s batch size (scaled from 128 to 1024 with proportional learning rate scaling), which is a standard practice that does not change model capacity or architecture.

III. LLM/VLM BENCHMARK METHODOLOGY

This section details our methodology for benchmarking Large Language Models (LLMs) and Vision-Language Models (VLMs) on the interior layout generation task. We prioritize reproducibility by defining strict schemas, standardized prompts, and unified output processing. All frontier models in Table II of the main paper are evaluated using the same pipeline described below.

A. Evaluated Models

We evaluate seven frontier models across text-only and vision-language (VLM) modes, plus two domain-specific baselines:

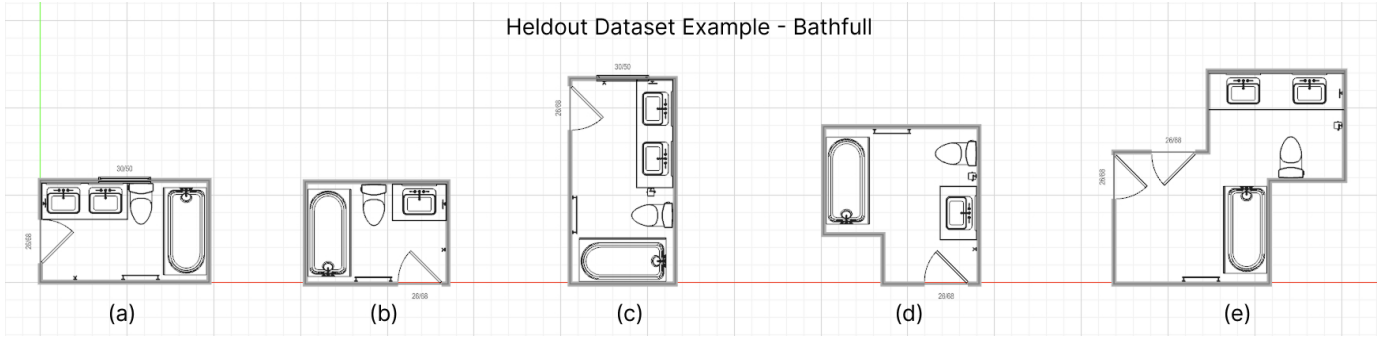


Fig. 2: Example heldout rooms for Full Bathrooms. From simple (a) to complex (e) room envelopes.

a) Frontier LLMs (Sub-group A1).:

- **Claude Opus 4.6, Sonnet 4.6, Haiku 4.5** (Anthropic) — all support both text and VLM modes; temperature 0.7, max tokens 4096.
- **GPT-5.2** (OpenAI) — text and VLM; temperature 0.7, reasoning effort `medium`.
- **Codex 5.3** (OpenAI) — text only (no vision support); temperature 0.7, reasoning effort `low`.
- **Gemini 3.1 Pro** (Google) — text and VLM; temperature 1.0 (provider-recommended).
- **Qwen 3.5** (via OpenRouter) — text and VLM; temperature 0.7.

b) Domain-Specific Methods (Sub-group A2).:

- **LayoutVLM** [3] — uses VLMs to generate spatial relations, then applies differentiable optimization. Requires a pre-specified furniture list as input, so it is excluded from Coverage evaluation. Evaluated using the authors’ released code with 3-trial best-of.
- **FlairGPT** [4] — GPT-based interior design generation with iterative refinement. Evaluated using the authors’ released code with 3-trial best-of.

All frontier models use the same unguided prompt template (Sec. III-D) with an entity catalog providing information parity with DDEP. Each model is run once per room (no best-of- k selection) to reflect realistic single-shot performance.

B. Heldout Dataset

We evaluate all methods on a heldout dataset of 50 individual rooms spanning 10 different room types: Master Bedroom, Bedroom, Full Bathroom, Kitchen, Living Room, Dining Room, Laundry Room, Powder Room, Office, and Garage. Each room type contains 5 rooms, providing a balanced representation across common residential spaces.

This heldout dataset was manually curated to include a diverse range of layout complexities, from simple rectangular rooms with minimal openings to more complex geometries with multiple doors, windows, and irregular shapes. Importantly, these rooms were not necessarily designed to be code-compliant or architecturally optimal; rather, they were intentionally selected to include edge cases and challenging scenarios that test the robustness of layout generation methods.

C. Data Representation

To enable fair comparison between domain-specific methods (DDEP, FlairGPT, LayoutVLM), text-based LLMs, and vision-based VLMs, we utilize a unified input schema, `LayoutGenerationInput`. This schema provides three concurrent representations of the input room:

- 1) **Structured Geometry:** A precise geometric definition including the room polygon, wall edges (with start/end coordinates and lengths), and openings (doors/windows with edge ID, parametric position, width, and swing direction). This is used for metric calculation and rendering.
- 2) **Text Description:** A natural language summary of the room geometry (area, perimeter, edge lengths, door/window locations) designed for text-only LLMs.
- 3) **Visual Representation:** A rendered floorplan image showing the room boundary, wall segments, openings, edge dimension labels, and a 10-ft grid overlay for scale reference. This allows VLMs to spatially ground their predictions.

A sample JSON input follows the structure below:

```

1 {
2   "version": "0.1",
3   "units": "feet",
4   "room": {
5     "polygon": [0.0, 0.0, 154.57, 0.0, 154.57, 87.20, 0.0, 87.20],
6     "edges": [
7       { "edge_id": 1, "length": 154.57,

```

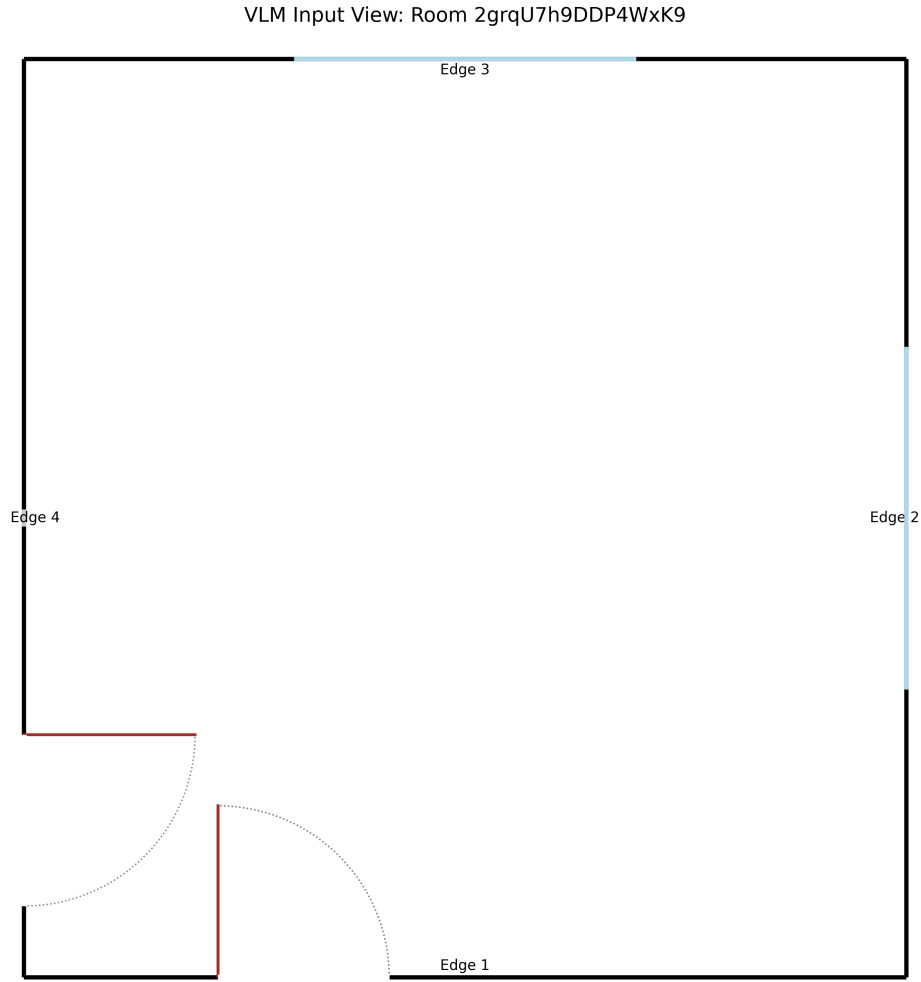


Fig. 3: Visual input sample for VLM benchmarking. The floorplan shows the room geometry with annotated edge IDs, dimension labels, and a 10-ft grid for spatial reference.

```

8      "start": [0.0, 0.0], "end": [154.57, 0.0] },
9      { "edge_id": 2, "length": 160.76,
10       "start": [154.57, 0.0], "end": [154.57, 87.20] }
11    ],
12    "openings": [
13      { "type": "DOOR", "edge_id": 2,
14        "t_start": 0.52, "t_end": 0.80,
15        "width": 2.5, "swing": "LEFT" }
16    ]
17  },
18  "text_description": "area=172.56 | perimeter=630.66 | ...",
19  "image_path": "images/2grqU7h9DDP4WxK9.png",
20  "inventory": [
21    { "category": "bed", "count": 1 },
22    { "category": "nightstand", "count": 2 }
23  ]
24 }

```

Listing 1: Sample LayoutGenerationInput

D. Prompt Engineering

We employ a two-stage prompting strategy consisting of a *System Prompt* and a *User Prompt*. Our prompt design evolved through three versions (v1–v3); all reported results use v3, which adds explicit coordinate system diagrams, wall condition guidance, and room-type-filtered entity catalogs.

1) *System Prompt: Role, Constraints, and Entity Catalog*: The System Prompt defines the agent’s role as an expert interior designer and enforces the output format. It contains four components:

a) *(a) Role and Guidelines.*: The model is instructed to “consider the room’s dimensions, shape, and door/window placements,” ensure minimum 36-inch circulation clearances, and avoid blocking doors.

b) *(b) Coordinate System.*: A critical component for spatial grounding. We include an ASCII diagram showing that room edges are numbered counter-clockwise (CCW) from the bottom edge, and define the parametric placement system:

$$P_{\text{entity}} = (\text{edge_id}, t_value, \text{lateral_offset}) \quad (27)$$

where edge_id is a 1-indexed wall identifier, $t_value \in [0, 1]$ is the normalized position along the edge (in the CCW direction), and lateral_offset is the perpendicular distance from the wall into the room (in inches). We explicitly state that $\text{distance} = t_value \times \text{edge_length}$ and provide concrete examples (e.g., “on edge 1 (150 inches), $t_value = 0.5$ means ~ 75 inches from the start”).

c) *(c) Wall Conditions.*: Each edge is annotated as INTERIOR (shared with another room) or EXTERIOR (outdoor-facing). We include placement guidance: beds prefer interior walls (quieter), TVs should face away from windows to avoid glare, and kitchen appliances go on interior walls (plumbing/electrical access).

d) *(d) Entity Catalog.*: To achieve information parity with DDEP, we provide a room-type-specific entity catalog listing valid furniture categories and their canonical dimensions. This catalog is loaded from the same DDEP vocabulary directory used during training, filtered to the target room type. For example, a MASTER_BED prompt includes king (76×80”), nightstand (24×24”), and dresser (60×24”), but omits kitchen-specific items like range or dishwasher. This prevents models from wasting capacity on irrelevant categories while matching the entity vocabulary available to DDEP.

2) *User Prompt: Room Context and Few-Shot Examples*: The User Prompt provides the specific room details:

- 1) **Room metadata**: type, bounding-box dimensions, area, door/window count, and enclosure type.
- 2) **Edge details**: For each edge, the prompt lists the edge ID, length in inches, and wall condition (INTERIOR/EXTERIOR). This enables the model to make spatially informed decisions (e.g., “Edge 1 is 154 inches long, fitting a king bed”).
- 3) **Image reference** (VLM mode only): The prompt notes that “a floorplan image is provided” showing room boundaries, door/window locations with edge ID labels, dimension labels, and a 10-ft grid reference.
- 4) **Text description** (always included): The structured text summary of room geometry.
- 5) **Few-shot example**: A room-type-matched JSON example showing the expected output format with realistic entity placements. For instance, the MASTER_BED example shows a king bed at edge 3 $t=0.5$, two nightstands at $t=0.15$ and $t=0.85$, and a dresser on edge 4.

All models operate in *unguided* mode: they select which entities to place based on room type and the entity catalog, rather than following a prescribed inventory list. This tests the model’s design judgment, not just its ability to execute instructions.

E. Output Processing and Free-Text Parsing

Raw LLM outputs require substantial post-processing before metrics can be computed. Our adapter pipeline implements a four-stage rule-based mapper, described below.

1) *Stage 1: JSON Extraction*: LLM responses frequently embed JSON within natural language explanations or markdown code blocks. We extract structured data using a three-tier strategy:

- 1) **Direct parse**: Attempt `json.loads()` on the full response.
- 2) **Markdown blocks**: Extract content from ````json ... ```` fences using the regex ````(?:json)?\s*({.*?})\s*````.
- 3) **Greedy brace match**: Find the outermost `{...}` substring and attempt parsing.

If all three stages fail, the response is marked as a parse failure and excluded from metrics (contributing to the success rate reported in Table II of the main paper).

2) *Stage 2: Entity Name Normalization*: LLMs generate diverse synonyms for the same furniture category. Our normalizer applies a deterministic mapping of 30+ variations to canonical DDEP categories:

LLM Variations	Canonical
queen_bed, king_bed, twin_bed, full_size_bed	bed
night_stand, bedside_table, night_table	nightstand
couch, loveseat, sectional	sofa
chest_of_drawers, bureau	dresser
end_table	side_table
dining_chair, armchair, recliner	chair
vanity, sink_base, counter	base_cabinet

Input names are lowercased, spaces and hyphens are converted to underscores, and both exact and substring matches are checked against the mapping table. Unrecognized names pass through unchanged.

3) *Stage 3: Anchor Type Enforcement*: DDEP places all entities using wall-anchored coordinates (`edge_id`, `t_value`, `lateral_offset`). Despite explicit prompt instructions, some LLMs occasionally produce free-standing placements with absolute (x, y) positions. Our adapter enforces wall-anchored placement through the following procedure:

- 1) **Explicit wall-anchored**: If the entity specifies `edge_id` and `t_value`, use them directly.
- 2) **Free-standing conversion**: If the entity provides a position $\mathbf{p} = (x, y)$ (either via an explicit `anchor_type: free_standing` field or inferred from the presence of position coordinates), we project $\mathbf{p}$ onto the nearest wall edge using vector projection:

$$t = \frac{(\mathbf{p} - \mathbf{s}) \cdot (\mathbf{e} - \mathbf{s})}{|\mathbf{e} - \mathbf{s}|^2}, \quad t_{\text{clamped}} = \text{clamp}(t, 0, 1) \quad (28)$$

where $\mathbf{s}$ and $\mathbf{e}$ are the edge start and end points. The closest point on the edge is $\mathbf{c} = \mathbf{s} + t_{\text{clamped}} \cdot (\mathbf{e} - \mathbf{s})$, and the lateral offset is $\|\mathbf{p} - \mathbf{c}\|$. We select the edge minimizing this distance. These conversions are logged as warnings and counted in the benchmark summary.

- 3) **Fallback**: If no valid placement can be determined, the entity is placed at $t=0.5$ on the longest wall edge.

4) *Stage 4: Entity Type Classification and Dimension Correction*: Each entity is classified as `PROP` (furniture/fixtures) or `CASEWORK` (built-in cabinetry) based on (1) an explicit `entity_type` field if present, or (2) keyword matching against casework indicators (`cabinet`, `counter`, `vanity`, `sink_base`).

LLMs sometimes output dimensions in feet instead of inches. We apply a heuristic conversion: if any dimension value is below 20, we multiply by 12 to convert to inches (furniture items are rarely smaller than 20 inches).

After the four-stage pipeline, each entity is validated and packaged into a `LayoutGenerationOutput` schema with metadata recording the raw response, parse statistics, any conversion warnings, and inventory compliance results (for guided mode).

IV. CONTROLLED SAME-DATA BENCHMARK DETAILS

This section expands the controlled same-data benchmark from Sec. IV-B of the main paper using the current metric bundle. All methods are rescored with the current definitions from Sec. I, including inward-offset door portals, doorway-validity gating, and the weighted overlap-clearance composite. In this section and in Table I of the main paper, “BLT” refers to the canonical unconditional BLT controlled bundle used in the shared benchmark harness.

A. Evaluation Universe

The controlled data split itself is unchanged: it is the same seed-17 split over five room types used in the original controlled benchmark. The benchmark metrics require valid room geometry and at least one door opening, so all methods are evaluated on the DDEP-valid subset, and direct cross-method comparisons use the room-id intersection where every method produced an output.

TABLE III: Controlled same-data evaluation universe.

Subset	Rooms	Note
Train split	15,740	Same frozen training split for all controlled methods.
Val split	2,009	Same frozen validation split for all controlled methods.
Test split	1,826	Original held-out test split before geometry filtering.
DDEP-valid benchmark subset	1,231	Test rooms with valid geometry and at least one usable door opening. All controlled metrics are computed on this subset or a subset thereof.
Shared comparison subset	1,225	Intersection across DDEP, ATISS, BLT, and the DDEP ablations. Table I in the main paper uses this subset.

The five room types are `BATHFULL`, `BEDROOM`, `LAUNDRY`, `LIVING`, and `MASTER_BED`. The split remains stratified by home, not by room, so all rooms from a given home stay in the same train/val/test partition. The benchmark reports results on the geometry-valid subset that can actually be scored by the wall-anchored metric pipeline.

B. Per-Room-Type Breakdowns on the Shared 1,225-Room Subset

TABLE IV: Per-room-type **Coverage** (%) on the shared 1,225-room controlled subset. Higher is better. Room counts by type are shown in the column headers.

Method	BATH (294)	BED (412)	LAUND (222)	LIVING (123)	MBED (174)
DDEP	62.7	65.7	27.8	60.1	47.3
ATISS	52.8	29.9	29.1	98.8	20.1
BLT	34.2	12.2	7.9	32.1	5.8

TABLE V: Per-room-type **Navigability** (Nav) on the shared 1,225-room controlled subset. Higher is better; the score range is $[-35, 100]$.

Method	BATH	BED	LAUND	LIVING	MBED
DDEP	-0.9	31.4	-12.3	29.2	24.8
ATISS	-35.0	-34.0	-35.0	-35.0	-4.3
BLT	-14.6	-15.6	-32.6	-25.6	-9.8

TABLE VI: Controlled same-data **overlap-clearance components** on the shared 1,225-room subset. EOF, GOA, DCI, and WBV are fractions; OC is the weighted composite reported in Table I of the main paper. Lower is better.

Method	EOF↓	GOA↓	DCI↓	WBV↓	OC(%)↓
DDEP	0.186	0.026	0.120	0.064	13.4
ATISS	0.004	0.000	0.009	0.035	0.5
BLT	0.708	0.007	0.006	0.056	35.7

The updated room-type breakdowns sharpen the interpretation of the controlled benchmark.

DDEP remains the only method with positive navigability on multiple room types. Under the doorway-validity rule, BEDROOM, LIVING, and MASTER_BED remain clearly reachable for DDEP, while BATHFULL and LAUNDRY are substantially harder because small door-clearance violations now invalidate entry rather than being hidden by large portal relocation.

ATISS can match inventory without producing usable entry geometry. The clearest example is LIVING, where ATISS reaches 98.8% coverage yet still averages -35.0 Nav. This is exactly the failure mode the updated metric is meant to expose: the inventory is present, but the room is not realistically enterable or traversable from the doorway.

BLT remains dominated by geometric collapse. Its doorway-intrusion term is small only because many placements never meaningfully interact with the doorway region. The dominant BLT error is still high entity overlap, reflected in $\text{EOF} = 0.708$ and $\text{OC} = 35.7\%$.

C. DDEP Ablation Details on the Full 1,231-Room DDEP-Valid Subset

TABLE VII: DDEP ablations on the full 1,231-room DDEP-valid controlled subset. SR = Success Rate, Nav = Navigability, EOF = entity-overlap fraction, and OC = weighted overlap-clearance composite. Higher is better for Coverage, SR, and Nav; lower is better for EOF and OC.

Ablation	n	Cov↑	SR↑	Nav↑	EOF↓	OC(%)↓
Main (full)	1,231	55.0	0.357	14.8	0.186	13.4
Wall-Only	1,231	50.9	0.319	10.0	0.170	12.7
No Continuous Groups	1,231	42.7	0.251	0.8	0.179	13.4
No Edge Coordinates	1,231	56.7	0.354	14.5	0.194	14.4
Order: t -value First	1,231	42.7	0.287	4.3	0.136	10.5
Order: edge, t -value Desc	1,231	46.6	0.310	8.0	0.177	13.3

Continuous spatial parameters remain the most important design choice. Removing the continuous groups drops DDEP from 55.0 to 42.7 coverage and from 14.8 to 0.8 Nav. This is still the largest single degradation among the ablations.

Explicit edge coordinates remain close to neutral on coverage and navigability, but not on geometric cleanliness. The no-edge-coordinate variant stays near the full model on Coverage and Nav, yet its OC rises from 13.4% to 14.4% and its EOF rises from 0.186 to 0.194. The model can often infer where to place furniture, but the explicit edge signal still helps keep the layout cleaner.

Token order affects both reachability and clutter. Putting t -value first produces lower Coverage and Nav than the full model. Its lower EOF and OC should not be read as an improvement in layout quality; they are largely a byproduct of sparser, less ambitious placements.

D. Production vs. Controlled DDEP Gap

The production DDEP model in the frontier benchmark achieves 98.1% Coverage and 0.86 SR on the 50-room heldout set, while the controlled same-data DDEP model in Table I of the main paper achieves 54.9% Coverage and 0.36 SR on the shared 1,225-room controlled subset. Both bundles are scored with the same metric definitions, so the remaining gap reflects model and data differences rather than a scoring mismatch.

- 1) **Training data scale.** Production trains on roughly 440K rooms, while the controlled joint model sees only 15.7K training rooms across the five controlled types.

- 2) **Room-type specialization.** Production uses room-type-specific models; the controlled benchmark trains a single joint model shared across five heterogeneous room distributions.
- 3) **Model capacity.** Production uses the larger deployed configuration, while the controlled model was intentionally reduced to stay closer to ATISS/BLT capacity.
- 4) **Training duration.** Production is trained substantially longer, with a more mature schedule and heuristics stack.
- 5) **Evaluation difficulty.** The stricter doorway-validity gate disproportionately hurts smaller rooms in the controlled suite, especially BATHFULL and LAUNDRY, where a single blocked vestibule can invalidate entry.

The controlled benchmark is therefore best read as a fairness protocol, not as DDEP’s best achievable performance. Its value is that every method is trained on the same data and rescored by the same geometry-aware pipeline.

V. TRAINING IMPLEMENTATION DETAILS

This section provides comprehensive implementation details for training the SBM encoder-decoder architecture, including architecture specifications, training configuration, loss functions, and infrastructure setup.

A. Transformer Backbone and Operating Modes

Given the encoder and decoder token embeddings

$$E^{\text{enc}} = \Phi_{\text{enc}}(X^{\text{enc}}) \in \mathbb{R}^{S_{\text{enc}} \times d}, \quad E^{\text{dec}} = \Phi_{\text{dec}}(X^{\text{dec}}) \in \mathbb{R}^{S_{\text{dec}} \times d},$$

we employ a standard Transformer encoder–decoder architecture. The same backbone is used in two operating modes: encoder-only (for room embeddings and retrieval) and encoder–decoder (for data-driven entity prediction, DDEP).

a) *Encoder-only: room embeddings and retrieval.*: In encoder-only mode, we use only the Transformer encoder. Given E^{enc} and an attention mask over non-padding tokens, we compute

$$M = \text{Enc}_{\theta}(E^{\text{enc}}) \in \mathbb{R}^{S_{\text{enc}} \times d},$$

where Enc_{θ} is a stack of standard self-attention and feed-forward blocks. We obtain a room embedding by pooling at the CLS position: $z(r_{\text{env}}) = M_0 \in \mathbb{R}^d$.

b) *Encoder–decoder: Data-Driven Entity Prediction (DDEP).*: In DDEP mode, the encoder processes the envelope producing memory M , held fixed during decoding. The decoder attends both to the prefix of generated tokens (causal self-attention) and to M (cross-attention): $H = \text{Dec}_{\theta}(E^{\text{dec}}, M) \in \mathbb{R}^{S_{\text{dec}} \times d}$.

We factorize the conditional distribution over contents as

$$p_{\theta}(Y \mid r_{\text{env}}) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, r_{\text{env}}),$$

with each $p_{\theta}(\cdot)$ parameterized by prediction heads applied to the decoder states in H .

B. Model Architecture

1) Encoder-Only Pathway:

- **Transformer Layers:** 8 layers with standard multi-head self-attention
- **Attention Heads:** 8 heads per layer
- **Token Embedding Dimension:** 512 (production) / 256 (controlled)
- **Feature Embedding Dimension:** 512 (production) / 256 (controlled)
- **Max Sequence Length:** 20 tokens
- **Dropout:** 0.2 for embeddings, 0.1 for attention layers
- **Activation Function:** GELU
- **Pooling Strategy:** CLS pooling at the first sequence position
- **Total Parameters:** ~25M (production) / ~6M (controlled)

2) Embedding Module: The embedding module handles mixed-type features (categorical and continuous):

- **Unified Mixed-Type Embeddings:** Processes both categorical and continuous features
- **Gated Feature Summation:** Enabled for combining feature groups with learnable gates
- **Continuous Group Aggregation:** Linear aggregation with MLP (dimension factor: 2)
- **Continuous Group Attention:** 4 attention heads for aggregating continuous feature groups
- **Positional Embedding:** Disabled (geometric features provide spatial information)

TABLE VIII: Complete encoder feature matrix specification. Each row corresponds to one feature in $X^{\text{enc}} \in \mathbb{R}^{F_{\text{enc}} \times S_{\text{enc}}}$. The “Applicable Tokens” column indicates which token types use each feature; inactive features are set to padding value -100 .

Feature Name	Type	Applicable Tokens
<i>Categorical Features</i>		
encoder_token_type_id	Categorical	All tokens
encoder_token_id	Categorical	All tokens
edge_id	Categorical	EDGE tokens
wall_condition	Categorical	EDGE tokens
<i>Continuous Scalar Features</i>		
area	Continuous	LAYOUT token
perimeter	Continuous	LAYOUT token
edge_length	Continuous	EDGE tokens
relative_length	Continuous	EDGE tokens
inside_thickness	Continuous	EDGE tokens
outside_thickness	Continuous	EDGE tokens
wall_opening_ratio	Continuous	EDGE tokens
t_value	Continuous	DOOR, WINDOW tokens
width	Continuous	DOOR, WINDOW tokens
wall_corners_distance_start	Continuous	DOOR, WINDOW tokens
wall_corners_distance_end	Continuous	DOOR, WINDOW tokens
<i>Continuous Group Features</i>		
edge_x1, edge_y1, edge_x2, edge_y2	Grouped	EDGE tokens

3) Encoder-Decoder Pathway (for DDEP):

- **Encoder:** Same architecture as encoder-only pathway
- **Decoder:** 8 transformer decoder layers with cross-attention to encoder memory
- **Causal Masking:** Decoder uses masked self-attention to prevent attending to future tokens
- **Shared MLP for Correlated Heads:** Optional shared MLP projection for correlated prediction heads (token_id, edge_id, t_value)
- **Total Parameters:** $\sim 50\text{M}$ (production) / $\sim 12\text{M}$ (controlled)

C. Tokenization and Feature Representation

1) *Complete Feature Tables:* Table VIII provides a complete listing of all encoder features with their types and applicable token types. Table IX provides the corresponding listing for decoder features.

D. Decoder Prediction Heads

Categorical Heads:

- **TokenHead:** Standard categorical head for token_id, rotation, discretized width. Architecture: $\text{Linear}(d \rightarrow d/2) \rightarrow \text{GELU} \rightarrow \text{Linear}(d/2 \rightarrow \text{vocab})$.
- **BinnedHead:** Binned prediction for edge_id, discretized t_value, discretized lateral_offset. Same architecture but outputs n_{bins} logits.

TABLE IX: Complete decoder feature matrix specification. Each row corresponds to one feature in $X^{\text{dec}} \in \mathbb{R}^{F_{\text{dec}} \times S_{\text{dec}}}$.

Feature Name	Type	Applicable Tokens
<i>Categorical Features</i>		
token_type_id	Categorical	All entity tokens (PROP, CASEWORK)
token_id	Categorical	All entity tokens
edge_id	Categorical	All entity tokens
rotation	Categorical (discretized)	PROP tokens only
width	Categorical (discretized)	CASEWORK tokens only
<i>Continuous Features</i>		
t_value	Continuous	All entity tokens
lateral_offset	Continuous	All entity tokens
vertical_offset	Continuous (optional)	All entity tokens
width	Continuous	PROP tokens (if not discretized)
height	Continuous (optional)	CASEWORK tokens
depth	Continuous (optional)	CASEWORK tokens

- **Independent Heads:** Separate heads for props vs. casework to allow different distributions.

Regression Heads:

- **RegressionHead:** $\text{Linear}(d \rightarrow d/2) \rightarrow \text{LeakyReLU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(d/2 \rightarrow 1)$. Predictions clamped to $[0, 1]$.
- **RegressionResidualHead:** Optional residual connection variant.

Shared MLP Projection: Optional shared MLP processes decoder output once for correlated heads. Correlated groups: $\{\text{token_id, edge_id, t_value}\}$ share the same MLP output.

E. Loss Functions

1) *Encoder-Only Composite Loss:*

$$\mathcal{L}_{\text{encoder-only}} = \beta_{\text{room}}\mathcal{L}_{\text{room}} + \beta_{\text{MTM}}\mathcal{L}_{\text{MTM}} + \beta_{\text{triplet}}\mathcal{L}_{\text{triplet}} + \beta_{\text{geom}}\mathcal{L}_{\text{geom}} \quad (29)$$

2) *Encoder-Decoder (DDEP) Loss:*

$$\mathcal{L}_{\text{DDEP}} = \mathcal{L}_{\text{encoder}} + \lambda_{\text{decoder}}\mathcal{L}_{\text{decoder}} \quad (30)$$

where $\mathcal{L}_{\text{decoder}} = \sum_{h \in \mathcal{H}_{\text{cat}}} w_h \mathcal{L}_{\text{CE}}^h + \sum_{r \in \mathcal{H}_{\text{reg}}} w_r \mathcal{L}_{\text{MSE}}^r$ and $\lambda_{\text{decoder}} = 1.0$.

a) *Encoder loss components.:* Room type classification ($\beta_{\text{room}} = 0.45$), masked token modeling ($\beta_{\text{MTM}} = 0.10$), graded triplet loss ($\beta_{\text{triplet}} = 0.45$, margin $m = 0.3$), and geometric preservation ($\beta_{\text{geom}} = 0.20$).

F. Training Strategy

a) *Production training (two-stage).:*

- **Stage 1 (Encoder Pre-training):** The encoder is trained on the full dataset of 439,932 samples across 53 room types using $\mathcal{L}_{\text{encoder-only}}$.
- **Stage 2 (DDEP Fine-tuning):** A separate encoder-decoder model is trained *per room type*, each on the samples for that type from the 75,720-sample subset spanning the 14 room types supported by DDEP. Each model’s encoder is initialized from the Stage 1 weights and fine-tuned; the decoder is trained from scratch. This per-type specialization allows each model to learn room-specific entity distributions and placement patterns.

Thus, the full 439,932-sample corpus is used for encoder pre-training, while autoregressive DDEP generation training uses only the 75,720-sample subset. All production DDEP results in the main paper correspond to models produced by this two-stage pipeline.

b) Controlled training (end-to-end).: For the controlled same-data benchmark (Table I in the main paper), we use a single-stage end-to-end training protocol:

- **Single-stage**: Both encoder and decoder are trained jointly from scratch using $\mathcal{L}_{\text{DDEP}}$. No separate encoder pre-training stage.
- **Data**: 15,740 training rooms across 5 room types (BATHFULL, BEDROOM, LAUNDRY, LIVING, MASTER_BED). One joint model serves all 5 types.
- **Model**: 256-dim hidden, 4 encoder layers, 4 decoder layers, 4 attention heads ($\sim 12\text{M}$ parameters encoder-decoder).
- **Duration**: 200 epochs with cosine annealing.
- **Seed**: Fixed to 17 (distinct from production seed 42) for all controlled runs.

The same controlled training protocol (data split, seed, and evaluation harness) is used for all five DDEP ablations. ATISS and BLT are trained on the same 15,740-room split using their respective architectures and hyperparameters (see Sec. II).

G. Training Hyperparameters

- **Optimizer**: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$)
- **Learning Rate**: 0.000345
- **Weight Decay**: 0.05
- **Max Gradient Norm**: 1.0
- **Batch Size**: 1024 (both production and controlled)
- **LR Schedule**: Cosine annealing with warmup ratio 0.1
- **Training Duration**: 2000 epochs (production) / 200 epochs (controlled)

H. Dataset Statistics

a) Production dataset.:

- **Training Samples**: 439,932 (augmented from 121,528 unique rooms)
- **Room Types**: 53 distinct types
- **DDEP Subset**: 75,720 samples across 14 room types
- **Triples**: 98,400 graded triplets

b) Controlled split.:

- **Training**: 15,740 rooms
- **Validation**: 2,009 rooms
- **Test**: 1,826 rooms total; 1,231 remain after door/geometry filtering for the benchmark metric pipeline, and the shared cross-method comparison in Table I of the main paper uses the 1,225-room intersection.
- **Room Types**: 5 (BATHFULL, BEDROOM, LAUNDRY, LIVING, MASTER_BED)
- **Split Strategy**: Stratified by home (seed 17), preventing train/test leakage
- **No augmentation**: The controlled split does not use data augmentation to maintain strict comparability

c) Data augmentation.: Rotation, X/Y mirroring, door jitter (range: 0.02, prob: 0.8), window jitter (range: 0.1, prob: 0.8), and window deletion (prob: 0.3).

I. Reproducibility

- **Random Seeds**: Production seed: 42. Controlled seed: 17. CUDA deterministic: enabled.
- **Environment**: Python 3.11, PyTorch 2.5.1, CUDA 12.4
- **Infrastructure**: NVIDIA H100 GPUs (80GB) via Modal cloud platform
- **Training Time**: ~ 3.5 days on H100 (production); ~ 8 hours on H100 (controlled, 200 epochs)

J. Model Complexity

- **Production**: $\sim 25\text{M}$ (encoder-only), $\sim 50\text{M}$ (encoder-decoder)
- **Controlled**: $\sim 6\text{M}$ (encoder-only), $\sim 12\text{M}$ (encoder-decoder)

VI. DDEP INFERENCE PIPELINE

The Data-Driven Entity Prediction (DDEP) service is deployed on AWS cloud infrastructure, utilizing a g5.2xlarge instance to leverage GPU acceleration for efficient model inference. The system operates via a RESTful API.

A. Data Transmission and Validation

The inference process begins on the client side, where a distilled version of the building data is extracted. This dataset contains essential BIM attributes, including room geometry, wall positions, and semantic labels, stripped of unnecessary rendering assets to minimize payload size. This distilled data is serialized and transmitted to the inference server.

Upon receipt, the server rigorously validates the incoming JSON payload against the `DDEPRequest` schema to ensure data integrity and type safety. Invalid requests are rejected immediately to prevent downstream processing errors.

TABLE X: Encoder-only ablation results under a fixed 200-epoch schedule. All variants reuse the reference hyperparameters. Absolute values are not directly comparable to the main paper tables because the production SBM model is trained longer and jointly with the decoder.

Variant	nDCG@5	Triplet	Entity Sim	NMI	ARI	Silhouette	Spearman
<i>Reference configuration</i>							
Reference (Enc-200ep)	56.7	100.0	81.4	0.640	0.307	0.152	0.021
<i>Loss component ablations</i>							
No MTM ($\beta_{\text{MTM}} = 0$)	57.7	100.0	81.4	0.640	0.307	0.152	-0.010
No Geometry ($\beta_{\text{geom}} = 0$)	57.4	99.9	81.3	0.640	0.307	0.152	0.038
No Triplet ($\beta_{\text{triplet}} = 0$)	63.6	48.7	80.7	0.640	0.307	0.034	0.012
No Room-Type ($\beta_{\text{room}} = 0$)	62.7	57.2	81.3	0.416	0.036	-0.494	0.000
Geom 10 $\times$ ($\beta_{\text{geom}} = 2.0$)	54.4	100.0	81.4	0.640	0.307	0.152	0.006
<i>Capacity and data scale</i>							
Small (4L / 4H / 256D)	66.7	100.0	81.4	0.709	0.379	0.161	0.167
50% Training Data	67.6	99.9	84.0	0.640	0.307	0.152	0.069

B. Inference Workflow

The core inference pipeline consists of three main phases:

- 1) **Normalization and Encoding:** The validated building data is converted into an internal representation. A normalization processor transforms the raw geometric coordinates and semantic features into a standardized feature space expected by the neural network. This includes scaling spatial dimensions and encoding categorical variables into tensor-compatible formats.
- 2) **Model Execution:** The normalized data is batched and fed into the DDEP model. The model utilizes an encoder-decoder architecture to generate a sequence of predictions. The inference engine runs on the GPU, producing raw output tokens that represent the type, location, and orientation of predicted entities within the room.
- 3) **Denormalization and Post-processing:** The raw model outputs are denormalized to map the predicted coordinates back to the original world space of the building. A post-processing module applies geometric heuristics to refine the placement, ensuring that predicted entities lie within valid room boundaries and do not violate physical constraints. Finally, the entities are packed into a response structure and returned to the client for instantiation in the 3D environment.

VII. ENCODER ABLATION STUDIES

This section reports encoder-only ablations of the SBM encoder. The goal is to understand the sensitivity of the embedding space to our loss design, architectural choices, and data scale under a fixed training recipe.

a) Scope and protocol.: All experiments in this section are encoder-only: we train the encoder for a fixed 200-epoch schedule and evaluate only the embedding metrics: type-constrained nDCG@5, triplet accuracy, entity-level similarity, and clustering/geometric correlation metrics (NMI, ARI, Silhouette, Spearman). Coverage, navigability, and OC are evaluated only for the full generative model and are *not* recomputed for these encoder-only variants.

Unless otherwise stated, all variants reuse the hyperparameters chosen via Bayesian optimization: learning rate 3.45×10^{-4} , weight decay 0.05, warmup ratio 0.1, dropout 0.2, and loss weights $\beta_{\text{room}} = 0.45$, $\beta_{\text{triplet}} = 0.45$, $\beta_{\text{geom}} = 0.2$, $\beta_{\text{MTM}} = 0.1$.

A. Ablation Configurations

We consider three groups of encoder variants:

- **Loss components:** removing each term in the composite loss, and one extreme geometry-weight setting ($\beta_{\text{geom}} = 2.0$).
- **Capacity:** a compact encoder with 4 layers / 4 heads / 256-dim vs. the reference 8L / 8H / 512-dim.
- **Data scale:** training on 50% of the data under the same schedule.

B. Results and Discussion

a) Loss components.: Removing MTM or geometry loss leaves metrics essentially unchanged. Removing triplet loss causes triplet accuracy to drop to 48.7% and Silhouette to deteriorate. Removing room-type supervision collapses type-structured organization (NMI 0.416 vs. 0.640, ARI 0.036 vs. 0.307, Silhouette -0.494), confirming that explicit class supervision is essential.

b) *Capacity and data scale.*: The compact 4L/4H/256D encoder achieves better retrieval and clustering than the reference (nDCG@5 66.7% vs. 56.7%, NMI 0.709 vs. 0.640). Training on 50% data yields strong performance with unchanged clustering, indicating the encoder is data-efficient under this schedule.

C. Embedding Metrics

Let $f(x) \in \mathbb{R}^d$ denote the encoder embedding of a room x , and let $s(x, y)$ be the cosine similarity $s(x, y) = \frac{f(x)^\top f(y)}{\|f(x)\| \|f(y)\|}$.

a) *Type-constrained nDCG@k.*:

$$\text{DCG}@k(q) = \sum_{i=1}^k \frac{2^{\text{rel}(q, r_{\pi_q(i)})} - 1}{\log_2(i+1)}, \quad \text{nDCG}@k(q) = \frac{\text{DCG}@k(q)}{\text{IDCG}@k(q)}. \quad (31)$$

b) *Triplet accuracy.*:

$$\text{TripletAcc} = \frac{1}{|\mathcal{T}|} \sum_{(q, p^+, p^-) \in \mathcal{T}} \mathbb{I}[s(q, p^+) > s(q, p^-)]. \quad (32)$$

c) *Entity-level similarity (Dice).*:

$$\text{Dice}_c(q, r) = \frac{2|E_c(q) \cap E_c(r)|}{|E_c(q)| + |E_c(r)|}, \quad \text{EntSim}(q, r) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \text{Dice}_c(q, r). \quad (33)$$

d) *Clustering: NMI.*:

$$\text{NMI}(Y, C) = \frac{I(Y; C)}{\sqrt{H(Y) H(C)}}. \quad (34)$$

e) *Clustering: ARI.*:

$$\text{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \frac{\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{N}{2}}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \frac{\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{N}{2}}}. \quad (35)$$

f) *Silhouette coefficient.*: $s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$, averaged over all rooms.

g) *Spearman rank correlation.*: $\rho_q = \text{corr}(R_s(q, \cdot), R_g(q, \cdot))$, averaged over queries.

D. Text Embedding Serialization Format

To evaluate text embedding baselines (E5-Large-v2, BGE-Large-en-v1.5, GTE-Large-en-v1.5), we serialize each room’s geometric metadata into a compact, label-free text string. The serialization is intentionally label-free — it excludes room type names and entity labels to prevent the text encoder from trivially solving retrieval via keyword matching.

The serialized format includes:

- **Room area** (normalized): area=0.342
- **Perimeter** (normalized): perimeter=0.518
- **Number of edges**: edges=4
- **Ordered segment lengths**: segments=[0.154, 0.161, 0.154, 0.161]
- **Door descriptors**: doors=(edge=1,u=0.52,w=0.08); (edge=3,u=0.3,w=0.06)

Fields are concatenated with | separators. All values are quantized to 3 decimal places for deterministic hashing. An example serialized string:

```
area=0.342 | perimeter=0.518 | edges=4 |
segments=[0.154,0.161,0.154,0.161] |
doors=(edge=1,u=0.52,w=0.08)
```

This format gives text encoders access to the same numeric room information available to SBM’s encoder (area, perimeter, edge lengths, door positions), ensuring a fair comparison of embedding quality. The key difference is that SBM processes these features through a structured tokenizer with typed embeddings, while text encoders must extract spatial structure from a flat string.

E. Cross-Type Semantic Retrieval

In the main paper we focus on type-constrained retrieval and clustering metrics. Here we provide cross-type retrieval evaluation, in which the candidate pool includes *all* room types simultaneously.

Method	nDCG	Prec(gr)	Triplet	Success	MRR
SBM (Ours)	63.6	58.7	51.9	92.0	79.5
E5-Large-v2	94.7	80.9	78.4	100.0	98.5
BGE-Large-en-v1.5	88.5	76.2	88.3	100.0	100.0
GTE-Large-en-v1.5	77.6	68.8	83.3	98.0	94.7

TABLE XI: Cross-type semantic retrieval performance @ $k = 5$. The candidate pool includes all room types. Text encoders dominate global semantic retrieval, while SBM maintains non-trivial cross-type performance despite being trained only on our BIM corpus.

a) Discussion.: Large text encoders substantially outperform SBM on cross-type semantic retrieval. E5 and BGE achieve very high nDCG@5 (94.7 and 88.5) and MRR (≥ 98.5). SBM, trained only on a few million BIM tokens, attains lower cross-type ranking quality (nDCG@5 63.6, MRR 79.5) but still retrieves relevant rooms for most queries (Success@5 92.0).

These results complement the main paper’s type-constrained and clustering evaluations, where SBM exhibits substantially better within-type structure, entity overlap, and geometric organization. Web-scale text encoders are preferable for tasks that prioritize coarse semantic similarity across room types, whereas SBM’s specialized embeddings better preserve room-type structure and geometry once the type is fixed.

VIII. CONSTRAINED DECODING VIA ARCHITECTURAL GRAMMARS

All DDEP results reported in the main paper and in this supplementary material use constrained decoding. We focus this section on the constrained-decoding mechanism itself rather than retaining an older with-vs.-without table computed under a different navigability definition.

A. Overview

Constrained decoding bridges the gap between a probabilistic sequence model and the hard structural requirements of architectural layouts. We treat decoding as a constraint satisfaction problem layered on top of the transformer’s predictive distribution: the model still proposes likely tokens, but invalid continuations are filtered before they can enter the final layout.

Scope. In this paper, “unguided” means that generation is not conditioned on external textual instructions, retrieval examples, or agent feedback. It does *not* mean unconstrained sampling. Constrained decoding is part of the DDEP inference pipeline in the same sense that a syntax checker is part of a code generator: it enforces validity while preserving the learned distribution over plausible completions.

Formally, for a token sequence $Y = \{y_1, \dots, y_T\}$, decoding is restricted to the subset satisfying a validity function

$$\mathcal{C}(Y) = 1,$$

where $\mathcal{C}$ is defined by architectural and code-compliance rules.

B. Architectural Grammar

We model the output space as a language generated by a hierarchical grammar:

- **Root:** $\text{Room} \rightarrow \{\text{Wall}\}^+ \times \{\text{Entity}\}^*$
- **Entity:** $\text{Entity} \rightarrow (\text{Type}, \text{Position}, \text{Dimensions}, \text{Orientation})$
- **Dependency:** $\text{Sink} \xrightarrow{\text{depends}} \text{Wall} \mid \text{Cabinet}$

This grammar ensures that generated layouts remain tied to the room perimeter and to valid architectural attachments. It also prevents impossible token sequences from propagating deep into the decode.

C. Constraint Categories

The validity function factorizes into functional and code-compliance constraints:

$$\mathcal{C}(Y) = \mathcal{C}_{\text{func}}(Y) \wedge \mathcal{C}_{\text{code}}(Y).$$

Functional constraints enforce room-appropriate usability, for example:

- wet-room fixtures should cluster along plausible plumbing walls,
- essential furniture must respect basic spatial adjacencies,

TABLE XII: Decoder token statistics by room type. “Decoder Tokens” counts content tokens (props + casework) excluding special tokens. **Note:** This table lists only the 14 room types (75,720 samples) used for production DDEP encoder-decoder training and evaluation; these samples are a subset of the larger 439,932-sample, 53-room-type corpus used for encoder pre-training. Room types marked with [†] are included in the controlled same-data benchmark (5 types).

Room Type	Decoder Tokens	Avg per Sample	Samples
BATHFULL [†]	119,127	8.20	14,541
BATHHALF	546	5.20	105
DINING	16,767	3.01	16,623
KITCHEN	439,672	11.34	12,289
OFFICE	921	4.51	204
PANTRY	2,439	5.31	459
BEDROOM [†]	43,063	3.99	10,777
GARAGE	9,960	4.71	2,115
LAUNDRY [†]	37,569	6.53	5,751
LIVING [†]	34,329	7.27	4,722
POWDER	2,976	5.22	570
WALK-IN CLOSET	5,763	8.39	687
WATER CLOSET	5,643	3.77	1,497
MASTER BEDROOM [†]	27,282	5.07	5,380
TOTAL	746,057	9.85	75,720

- attached casework must remain consistent with the wall-anchored representation.

Code-compliance constraints enforce architectural validity, for example:

- minimum egress width near doors,
- protection of hinged-door swing regions,
- rejection of placements that violate room bounds or create impossible clearances.

D. Practical Role in the Benchmark

Constrained decoding matters because the evaluation metrics are strict about physical entry and traversability. The navigability metric begins at an inward-offset portal and invalidates doors whose immediate vestibule is blocked; the overlap-clearance metric separately measures door-clearance intrusion. In that setting, constrained decoding is not a cosmetic post-process. It directly reduces impossible layouts before they reach metric evaluation.

The main paper therefore reports DDEP only in its constrained-decoding configuration. This is the production-faithful inference path and the one that should be compared against ATISS, BLT, and the frontier baselines.

IX. DECODER TOKEN STATISTICS BY ROOM TYPE

Table XII summarizes the distribution of decoder tokens per room type. Decoder tokens SOS and EOS are excluded; only content tokens corresponding to props and casework entities are included. DDEP currently supports the set of room types listed in the table; all layout generation and evaluation experiments in the main paper are drawn from this inventory.

X. ADDITIONAL QUALITATIVE RESULTS

TABLE XIII: **Prior-Generation Frontier LLM Benchmark.** Results from prior-generation frontier LLMs on the same 50-room held-out set used in the main paper (Table II). We report mean $\pm$ std. Higher is better for Coverage and Navigability (mean and SR); lower is better for OC, DF, and latency. OC reports the weighted overlap-clearance composite from Sec. 1. All methods use the same unguided prompt with entity catalog (information parity). [†]GPT-5.2 achieved only 46% success rate due to reasoning token exhaustion.

Method	Coverage	Navigability			OC (%)	Latency (s)
	$\uparrow$ mean $\pm$ std	$\uparrow$ mean $\pm$ std	$\uparrow$ SR	$\downarrow$ DF	$\downarrow$ mean $\pm$ std	$\downarrow$ mean $\pm$ std
<i>Text-based methods</i>						
Claude Opus 4.5	75.7 $\pm$ 18.6	39.0 $\pm$ 57.5	0.54 $\pm$ 0.45	0.43 $\pm$ 0.44	6.7 $\pm$ 11.5	13.0 $\pm$ 2.7
GPT-5.2 [†]	58.5 $\pm$ 20.0	69.7 $\pm$ 45.9	0.78 $\pm$ 0.36	0.23 $\pm$ 0.32	9.3 $\pm$ 10.8	40.3 $\pm$ 13.4
Claude Sonnet 4.5	74.6 $\pm$ 16.4	50.2 $\pm$ 55.9	0.62 $\pm$ 0.44	0.33 $\pm$ 0.40	9.4 $\pm$ 12.6	–
Codex 5.2	67.0 $\pm$ 15.0	62.8 $\pm$ 50.3	0.72 $\pm$ 0.39	0.27 $\pm$ 0.36	5.4 $\pm$ 8.2	7.9 $\pm$ 3.1
Gemini 3 Pro	64.0 $\pm$ 15.0	67.5 $\pm$ 48.5	0.76 $\pm$ 0.38	0.25 $\pm$ 0.34	6.5 $\pm$ 10.0	24.3 $\pm$ 20.6
Claude Haiku 4.5	76.7 $\pm$ 13.1	42.0 $\pm$ 61.1	0.56 $\pm$ 0.46	0.40 $\pm$ 0.46	15.7 $\pm$ 15.9	–
<i>VLM methods</i>						
Claude Opus 4.5	77.2 $\pm$ 13.1	65.7 $\pm$ 49.6	0.73 $\pm$ 0.39	0.21 $\pm$ 0.34	6.0 $\pm$ 10.7	13.5 $\pm$ 2.2
Claude Sonnet 4.5	75.1 $\pm$ 16.4	58.1 $\pm$ 52.4	0.68 $\pm$ 0.41	0.28 $\pm$ 0.38	10.6 $\pm$ 13.4	–
Gemini 3 Pro	65.3 $\pm$ 15.0	68.2 $\pm$ 49.6	0.77 $\pm$ 0.39	0.24 $\pm$ 0.35	12.5 $\pm$ 14.0	18.9 $\pm$ 23.0
Claude Haiku 4.5	72.8 $\pm$ 17.8	40.8 $\pm$ 58.8	0.55 $\pm$ 0.45	0.40 $\pm$ 0.44	19.4 $\pm$ 19.4	–
<i>Domain-specific methods</i>						
DDEP (Ours)	98.1 $\pm$ 6.3	79.2 $\pm$ 44.7	0.86 $\pm$ 0.34	0.20 $\pm$ 0.32	1.9 $\pm$ 3.1	3.18 $\pm$ 0.18

Heldout Living Room Dataset



Fig. 4: Layouts for Living areas by the evaluated methods (part 1).

Heldout Living Room Dataset



Fig. 5: Layouts for Living areas by the evaluated methods (part 2).

Controlled DDEP and baseline diagnostics (A)

Controlled DDEP examples are shown on the most interpretable heldout rooms; predicted furniture is summarized below each plot.

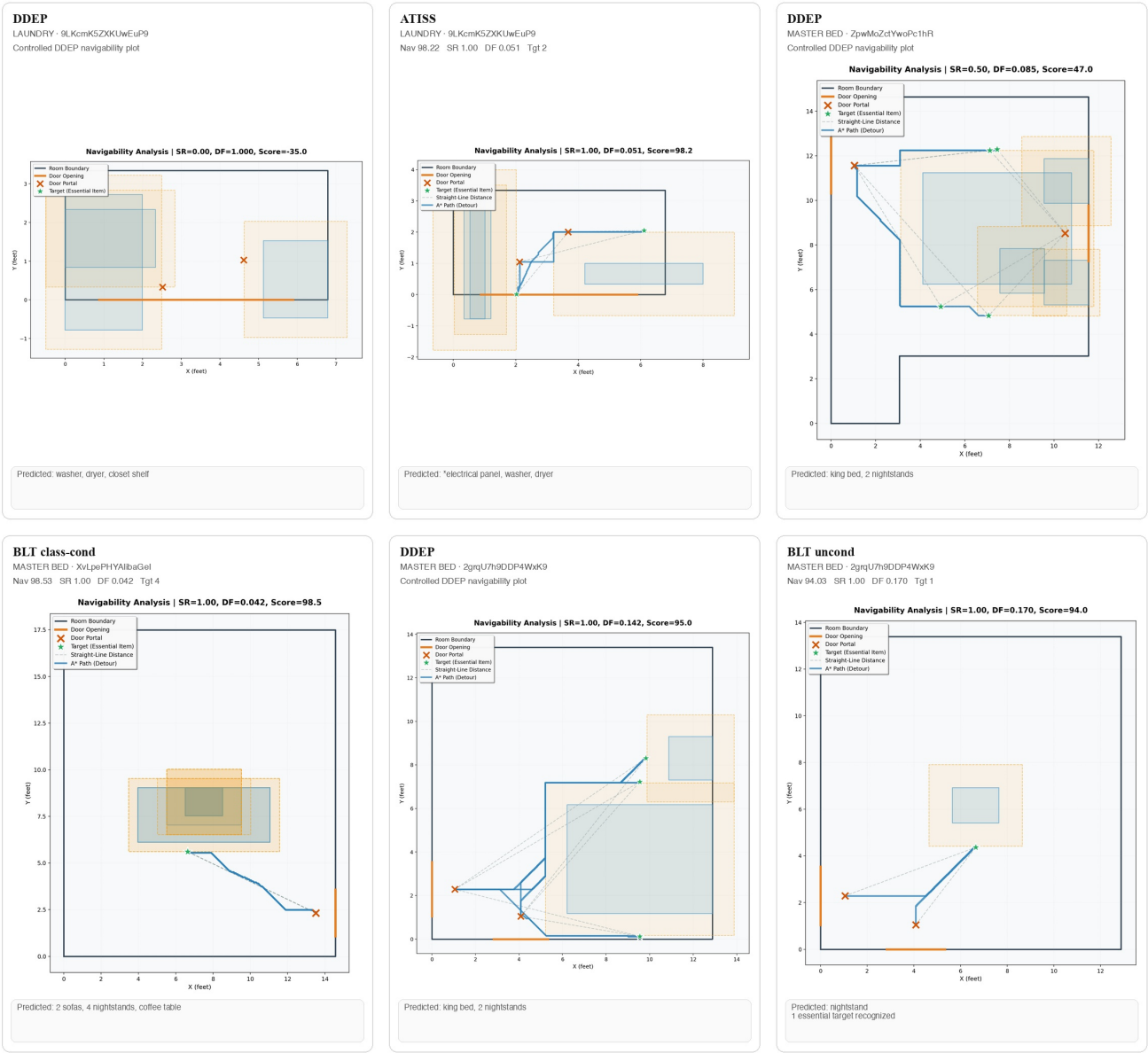
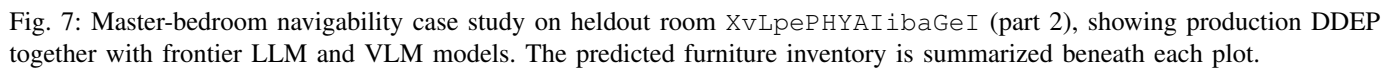


Fig. 6: Controlled DDEP and baseline navigability diagnostics (part 1). Controlled DDEP examples are shown on the most interpretable heldout rooms, and the predicted furniture inventory is summarized beneath each plot.

Heldout room XvLpePHYAlibaGeI · MASTER_BED · predicted furniture is summarized below each plot.



DDEP Generated Layouts (not part of training dataset). NOT SCALED

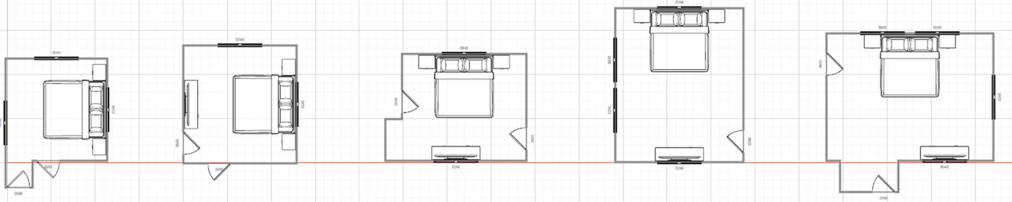
Bedroom



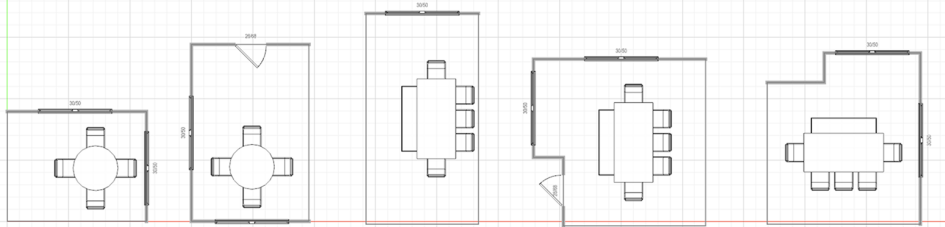
Kitchen



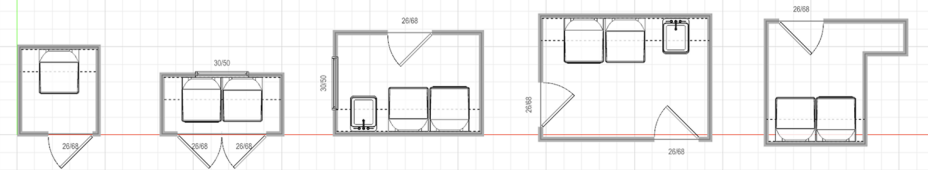
Master Bedroom



Dining



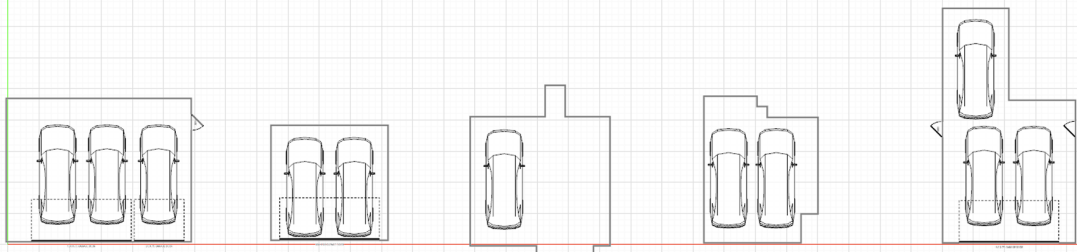
Laundry



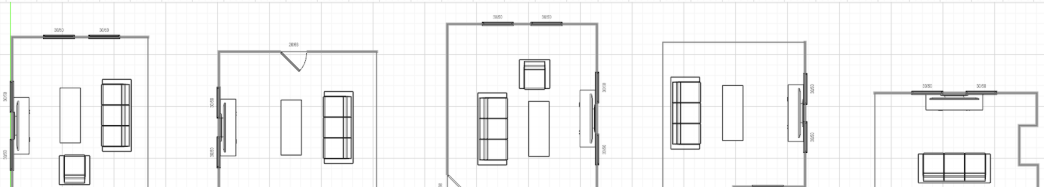
Office



Garage



Living



REFERENCES

- [1] D. Paschalidou, A. Kar, M. Shugrina, K. Kreis, A. Geiger, and S. Fidler, “Atiss: Autoregressive transformers for indoor scene synthesis,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [2] X. Kong, L. Jiang, H. Chang, H. Zhang, Y. Hao, H. Gong, and I. Essa, “BlT: Bidirectional layout transformer for controllable layout generation,” in *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XVII*, (Berlin, Heidelberg), p. 474–490, Springer-Verlag, 2022.
- [3] F.-Y. Sun, W. Liu, S. Gu, D. Lim, G. Bhat, F. Tombari, M. Li, N. Haber, and J. Wu, “Layoutvlm: Differentiable optimization of 3d layout via vision-language models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 29469–29478, 2025.
- [4] G. Littlefair, N. S. Dutt, and N. J. Mitra, “Flairgpt: Repurposing llms for interior designs,” 2025. EUROGRAPHICS 2025.